2. Install the EF Core NuGet packages.
3. Design your app's `DbContext` and entities that make up your data model.
4. Register your app's `DbContext` with the ASP.NET Core DI container.
5. Use EF Core to generate a migration describing your data model.
6. Apply the migration to the database to update the database's schema.

This process may seem a little daunting already, but I'll walk through steps 1-4 in sections 12.2.1-12.2.3 and steps 5-6 in section 12.3, so it won't take long. Given the space constraints of this chapter, I stick to the default conventions of EF Core in the code I show you. EF Core is far more customizable than it may initially appear to be, but I encourage you to stick to the defaults wherever possible, which will make your life easier in the long run.

The first step in setting up EF Core is deciding which database you'd like to interact with. It's likely that a client or your company's policy will dictate this decision, but giving some thought to it is still worthwhile.

12.2.1 Choosing a database provider and installing EF Core

EF Core supports a range of databases by using a provider model. The modular nature of EF Core means that you can use the same high-level API to program against different underlying databases; EF Core knows how to generate the necessary implementation-specific code and SQL statements.